Random Decision Forests

DSC 340 Applied Machine Learning

March 14

The Plan

- Random (decision) forests
- Ensemble methods: bagging and boosting

Random Decision Forests

Recap: Decision Trees (CART)

- At each node, split based on the value of just one feature.
- Leaves correspond to decisions: assignment to most likely class.
- Can also be used for regression (at leaves, output mean, not mode).
- Intuitive and explainable. How humans think we categorize/diagnose.
- Highly expressive model: can learn ANY data set if tree big enough.
- Major drawback: Highly prone to overfitting.
- Can constrain size of tree (hyperparameter). Pruning methods exist.
- However, these are workarounds. The problem of overfitting persists.

Random Decision Forests

- Tin Kam Ho (1995) [Chinese: 何天琴]
- Leo Breiman (GINI)
- Adele Cutler







Big Idea

- Instead of one big tree, **many small trees**
- Small trees trained on only a **subset of training data**
- And/or: Trained taking only a **subset of features** into account
- **To make a prediction, we consult all the trees** for their predictions
- In effect, **each little tree gets a vote** on the class label
- **Binary classification, majority vote** "wins": forest reports that label
- **Multiclass: plurality vote**, i.e., output the **mode** (most popular label)
- **Regression:** output the **mean** (**average** prediction for target value)

But … Occam's Razor?

- As models become more complex, usually generalize more poorly.
- Certainly true for **trees**: big trees are not very **accurate** on test data.
- **Bias-variance tradeoff** that we have seen before.
- Decision trees: Low bias, but very high variance (overfit).
- But in random forest, as forest grows, **resists overfitting.**
- Downside: no longer easy to explain: black box (vs. glass box) model.
- (Methods do exist to learn **Born-Again Tree Ensembles** from forest.)

Sampling the training data (sample the rows)

- Suppose N training examples.
- Sample B "**bags**" of data from training set.
- Each bag has $n \ll N$ examples (rows).
- Sampling is "**with replacement**."
  - Probability theory: can sample same point/row more than once.
  - Example: as we repeatedly roll a die, we sample {1, 2, …, 6} with replacement.
- **Train each tree on just one bag, not whole training set.**
- Since it doesn't "see" the whole training set, it doesn't overfit to it.
- Similar to k-Fold C.V. but the algorithm does the sampling for you.

Random Subspace Method (sample the columns)

- Tin Kam Ho (1998).
- Suppose we have M features.
- **Randomly sample** $m < M$ features, with replacement.
- Linear algebra: **projects** training data down to fewer dimensions.
- Can think of as assigning a 0 to the excluded columns, or dropping.
- Transformed space is said to be a **random subspace** of **feature space**.
- Can be used together with, or instead of, sampling from training set.*
- Again, doesn't "see" whole picture, so more resistant to overfitting.

*Note: sklearn only samples columns unless you hyperparameter `bootstrap=True`

Quote from paper

"My method relies on an **autonomous, pseudorandom procedure** to **select a small number of dimensions** from a given **feature space**. In each pass, such a selection is made **and a subspace is fixed** where all points have a constant value (say, zero) in the unselected dimensions. **All samples are projected to this subspace**, and a decision tree is constructed **using the projected training samples**.

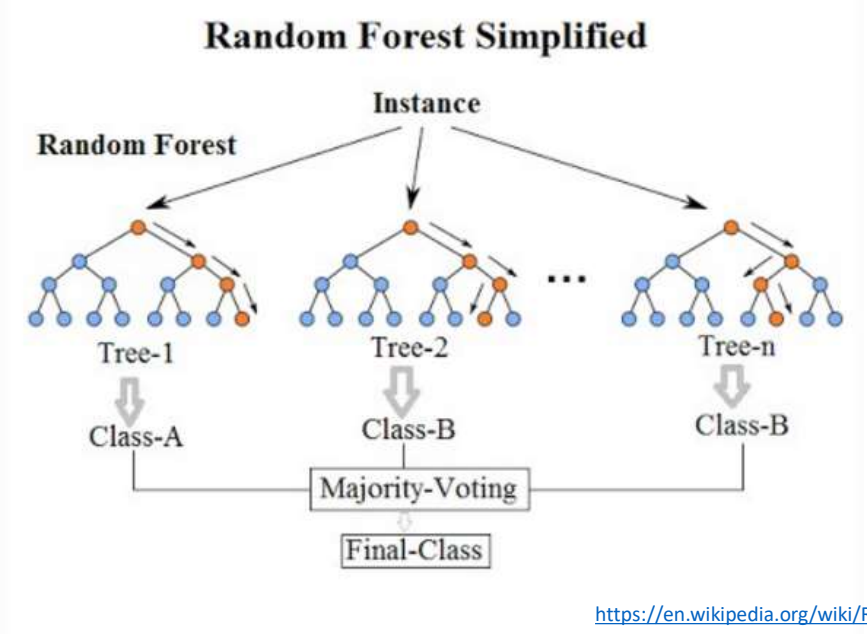
"**In classification** a sample of an unknown class **is projected to the same subspace** and **classified using the corresponding tree**."

– Tin Kam Ho

"The Random Subspace Method for Constructing Decision Forests" (IEEE, 1998)

Voting metaphor

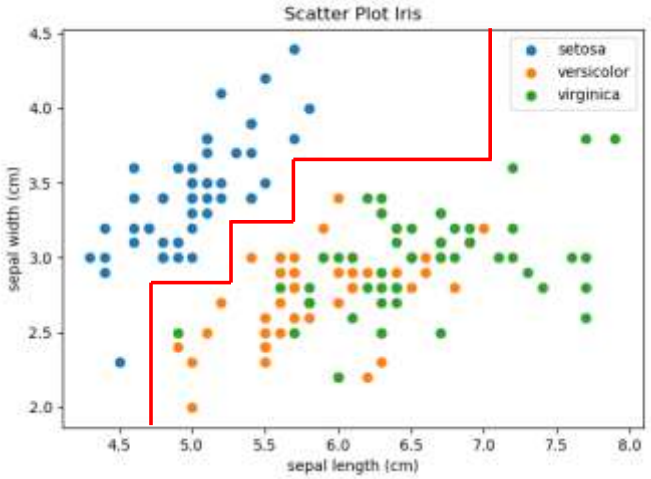
- Suppose we have n team members.
- Each one is more familiar with different aspects of the project.
- Each one has specialized skills that other team members lack.
- If we ask for recommendation, their recommendations may differ.
- However, can aggregate their decisions through some kind of vote.
- We expect prediction of team to be better than that of any individual.
- **Classification: mode** (plurality or maybe majority). **Regression: mean**.
- (Aside: Interested in voting theory? See Prof. Bailey.)



https://en.wikipedia.org/wiki/Random_forest

Choice of split

- Last time: split at axis.
- Example: $x_1 > 5.5$
- May need MANY splits!
- Other methods exist …



Splitting methods

- **Axis-parallel linear splits** (discussed last time)
- **Oblique linear splits**
  - Linear combination of features: not necessarily an axis.
  - Hyperplane that partitions the remaining data into 2 groups.
  - Bounded by hyperplanes but not necessarily axes (polytope, not rectangular).
- **Piecewise linear splits**
  - Voronoi tessellation of the feature space.
  - Relies on Euclidean or Manhattan distance (etc.) from some point.
  - Space partitioned based on distance to examples (analogous to kNN).
- Many other methods: **clustering**, **class centroid**, **perceptron**, etc.

Voronoi diagram



Bagging

- **Ensemble methods** in machine learning
- General method: can be used for **any** ML model, not just trees!
- One approach is **bagging** = **bootstrap aggregating**
- **Train B models** separately (perhaps **different** methods; should **vary**)
- Suppose $y_b(\mathbf{x})$ is the **prediction** of model b where $b \in \{1, 2, \ldots, B\}$
- Then the prediction of **committee** (bag) for feature vector $\mathbf{x}$ is:

$$y_{\text{committee}} = \frac{1}{B}\sum_{b=1}^{B} y_b(\mathbf{x})$$

- This is regression: again, for classification, output the **mode**.

Boosting

- Another ensemble method.
- **AdaBoost** = **Adaptive Boosting**
- **Many weak learners:** each may only be slightly better than chance.
- Predictions combined through **weighted majority scheme**.
- **Points misclassified** by any model given **more weight** in next round.
- Many other ensemble methods exist.
- Note that this is NOT same as grid search and choosing best model.
- (Story: SAT solver competition.)

Parameters and Hyperparameters

- **Parameters**:
  - Trees with their nodes: variable and value
- **Hyperparameters**:

	using sklearn:
• Number of trees	`n_estimators`
• Number of samples or size B of bag	`bootstrap=True` `max_samples`
• Number of features to consider	`max_features`
• Splitting method (GINI, entropy, etc.)	`criterion`

  - Max depth, min_samples (all the CART hyperparameters)
- Neither parameters nor hyperparameters: n_jobs, `random_state`, etc.

Assessment

- Less supervision from you (hyperparameters) than decision trees
- Does not necessarily require normalization or scaling (but can)
- Highly resilient for problems with missing data
  - Uses mode or mean imputation
  - Split then effectively ignores the missing value
- Fast
- Often the most accurate of all classification methods
- Doesn't handle regression as well as other methods (discontinuous)
- Not as intuitive to understand or explain to others

To Learn More

- See information on scikit-learn site:
  - https://scikit-learn.org/stable/modules/ensemble.html#forest
  - https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_iris.html
  - https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
- Two videos posted to Moodle (soon!)
  - Overview
  - Tutorial using Python (reviews many other concepts)